AROR UNIVERSITY

LEARN • INNOVATE • LEAD

"Building Intelligent Tomorrows"

ARI-406 | GENERATIVE AI COURSE

LECTURES 10–12

Generative AI

From Attention to Transformer Architectures

Attention
Limitations of recurrent models; Query/Key/Value; attention scores; scaled dot-product attention; self-attention

Transformer Components
Multi-head attention; feed-forward networks; residual connections; normalization; positional information

Transformer Architectures
Encoder-only models; decoder-only models; encoder-decoder models; causal attention; Transformer inference



Text
Write & generate content

Images
Create visuals

Code
Build solutions

Audio
Compose sounds

Video
Bring ideas to life

Create • Innovate • Transform

COURSE INSTRUCTOR

Dr. Rajesh Kumar

Assistant Professor
University of New York Tirana

Adjunct Faculty Member
AROR University

Generative AI

- ATTENTION

Lecture Objectives

- **Learning Objectives**
- By the end of this lecture, you should be able to:
- Explain limitations of recurrent models
- Describe the purpose of attention
- Explain **Query, Key, and Value**
- Describe how attention scores are calculated
- Explain scaled dot-product attention
- Describe self-attention
- Trace the complete attention process

Before Transformers: Recurrent Models

- **Recurrent Models**
- Traditional sequence models include:
- **RNNs → LSTMs → GRUs**
- They process a sequence step-by-step:
- $\text{Token}_1 \rightarrow \text{Token}_2 \rightarrow \text{Token}_3 \rightarrow \text{Token}_4 \rightarrow \ldots$
- Information is passed from one step to the next.

Limitations of Recurrent Models

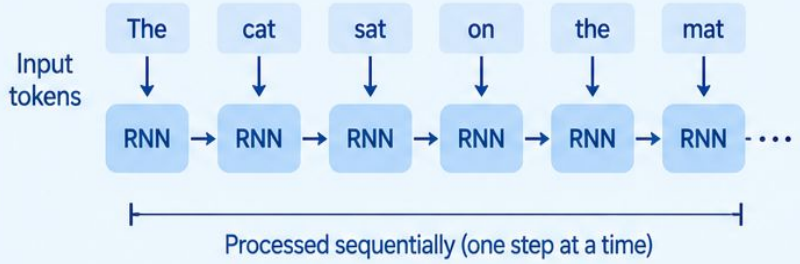
- **Why Is Recurrence Difficult?**
- **1. Sequential processing**
- Tokens are processed one after another.
- **2. Long-range dependencies**
- Earlier information can be difficult to preserve.
- **3. Limited parallelization**
- Later steps depend on earlier steps.
- **4. Long sequences**
- Information must travel through many steps.

Limitations of Recurrent Models

Why Is Recurrence Difficult?

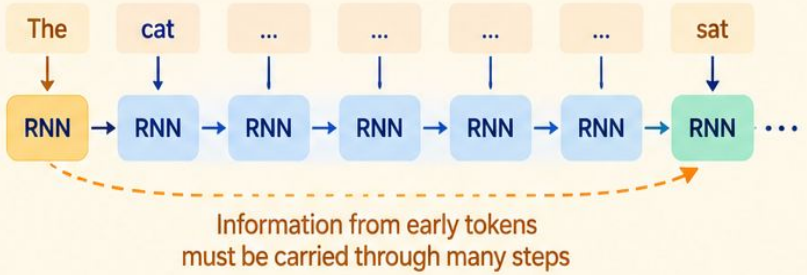
1 Sequential processing

Tokens are processed one after another.



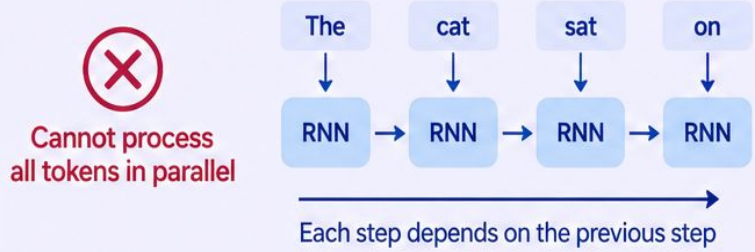
2 Long-range dependencies

Earlier information can be difficult to preserve.



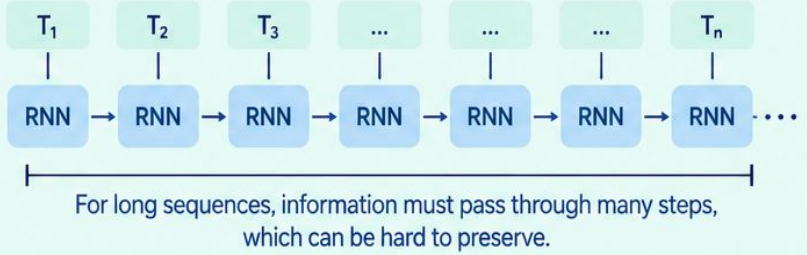
3 Limited parallelization

Later steps depend on earlier steps.



4 Long sequences

Information must travel through many steps.



Key Takeaway: Recurrent models process tokens sequentially, which makes it hard to capture long-range dependencies and limits parallelization — especially for long sequences.

Activity

- **What Should We Focus On?**
- Consider:
- **“The animal didn't cross the street because it was too tired.”**
- **Question**
- What does **“it”** most likely refer to?

WHAT SHOULD 'IT' FOCUS ON?



A animal

B street

C tired

Most relevant earlier word

Activity Answer

- ✅ **Answer**
- “The **animal** didn't cross the street because **it** was too tired.”
- **“it” → “animal”**
- **Key Idea**
- Not every token is equally relevant.
- **Attention helps the model focus on important tokens.**

What Is Attention?

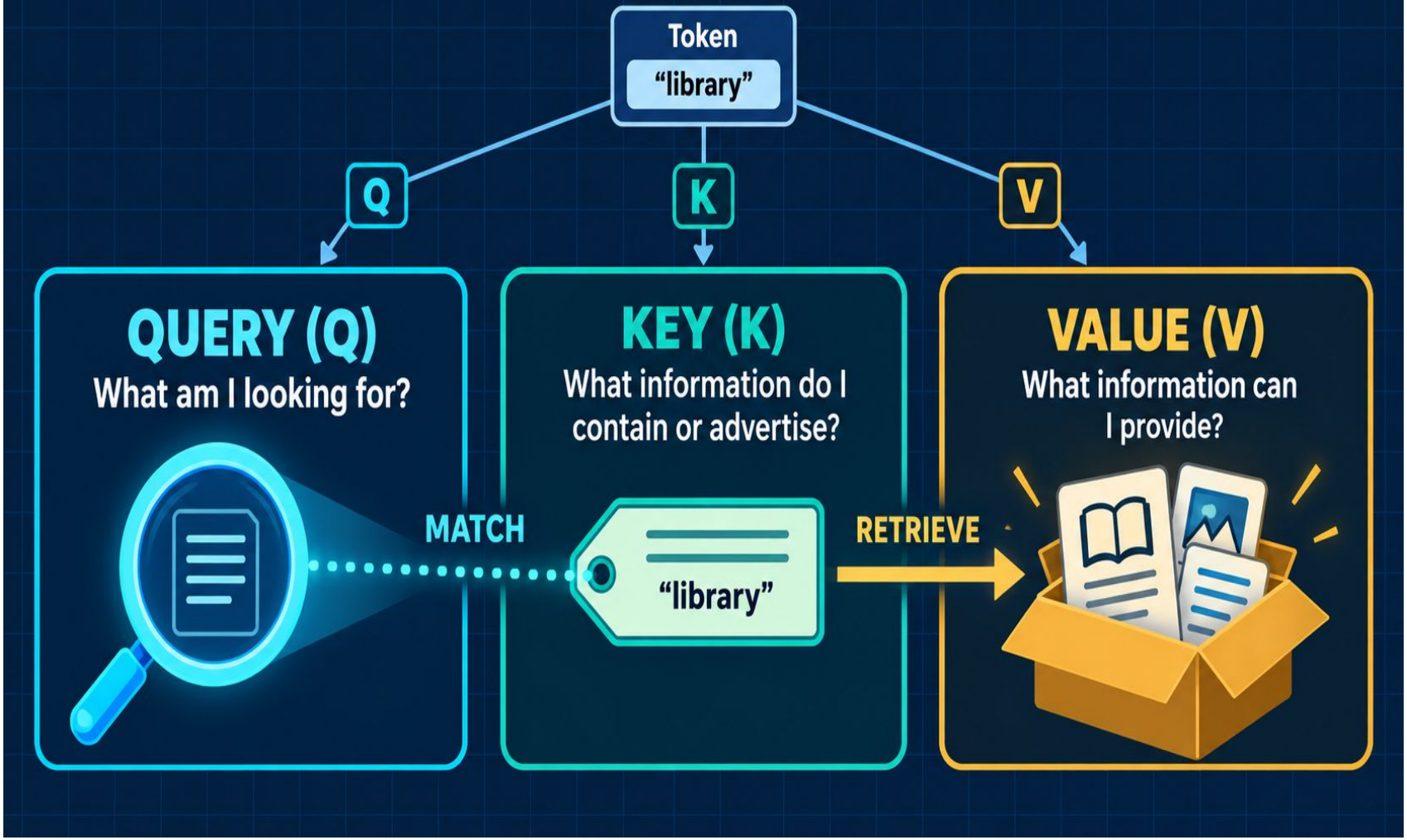
- **Attention**
- Attention allows a model to:
- **Focus on relevant parts of the input when processing a token.**
- Instead of treating every token equally:
- Token_1 Token_2 Token_3 Token_4
- the model learns:
- **Which tokens matter more?**

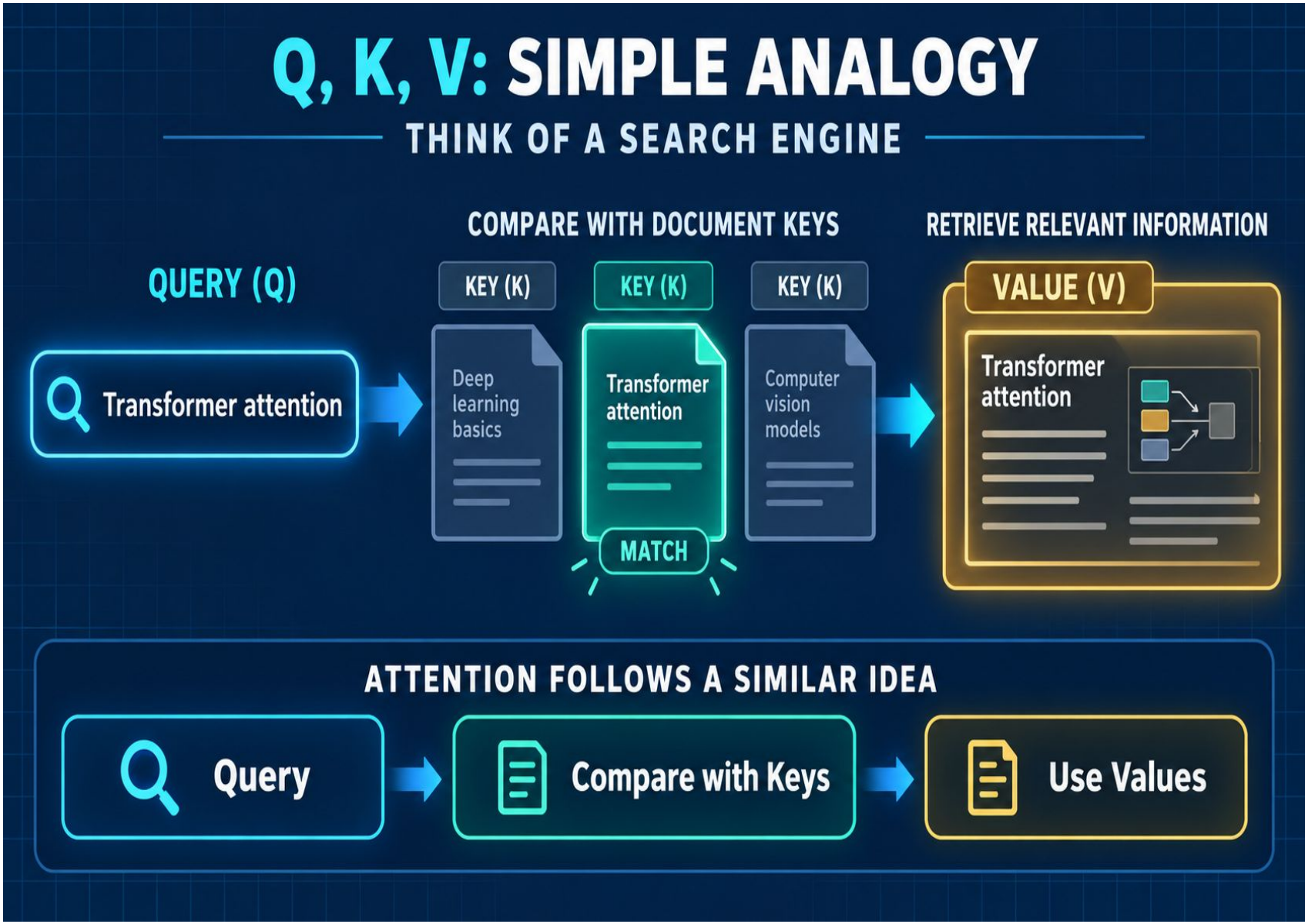
Query, Key, and Value

- **Three Important Concepts**
- Each token produces:
- 🔍 **Query (Q)**
- **What am I looking for?**
- 🏷️ **Key (K)**
- **What information do I contain / advertise?**
- 📦 **Value (V)**
- **What information can I provide?**

QUERY, KEY, AND VALUE

Each token produces three representations




Q, K, V: SIMPLE ANALOGY
THINK OF A SEARCH ENGINE
QUERY (Q)
Transformer attention
COMPARE WITH DOCUMENT KEYS
KEY (K)
KEY (K)
KEY (K)
Deep learning basics
Transformer attention
Computer vision models
MATCH
RETRIEVE RELEVANT INFORMATION
VALUE (V)
Transformer attention
ATTENTION FOLLOWS A SIMILAR IDEA
Query
Compare with Keys
Use Values

Activity 2: Q, K, or V?

- 🎯 **Activity — Match the Concept**
- Match each description:
- **1. Query**
- **2. Key**
- **3. Value**
- with:
- **A. Information that can be retrieved**
- **B. What the current token is looking for**
- **C. Information used to determine whether there is a match**

- **Activity 2: Answer**
- ✅ **Answer**
- **Query → B**
- What am I looking for?
- **Key → C**
- Does this token match what I need?
- **Value → A**
- What information does this token provide?
- **Remember**
- **Q searches → K matches → V provides**

- **From Attention Scores to Attention Weights**
- **Attention Scores Are Not the Final Output**
- After comparing the **Query** with each **Key**:
- **Query · Key → Attention Score**
- The scores are then:
- **1. Scaled**
  2. Passed through Softmax
  3. Converted into Attention Weights

Why Scale the Attention Scores?

- For large vectors, dot products can become **very large**.
- Large scores can make softmax:
- Too sharp
- Less stable
- Harder to train effectively
- **Solution**
- Divide by:
- $\sqrt{d_k}$
- where d_k **= dimension of the Key vectors**
- **Key Idea**
- **Scaling keeps attention scores in a manageable range.**

Scaled Dot-Product Attention

- **Scaled Dot-Product Attention**
- The complete formula is:
- **Attention(Q, K, V) = softmax(QK^T / $\sqrt{d_k}$)V**
- **Step by Step**
- **QK^T**
  → Calculate similarity scores
- **÷ $\sqrt{d_k}$**
  → Scale the scores
- **Softmax**
  → Convert to attention weights
- **× V**
  → Combine relevant information

Activity — Attention Pipeline

- ut these steps in the correct order:
- **A. Apply Softmax**
- **B. Multiply by Values**
- **C. Compare Query with Keys**
- **D. Scale the scores**
- **Your Answer**
- ___ → ___ → ___ → ___

Answer

- The correct order is:
- **Compare Query with Keys → Scale → Softmax → Multiply by Values**

From Scores to Weights

Softmax converts raw attention scores into normalized attention weights.

1 Raw Attention Scores

The model computes a score for each token by comparing the Query with the Key.

Token	Score (Query · Key)
The	1.0
cat	3.0
sat	0.5

2 Apply Softmax

Convert scores to probabilities using the softmax function:

$$\text{softmax}(s_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

This converts the scores into normalized attention weights that sum to 1.

3 Attention Weights

After applying softmax:

Token	Attention Weight
The	0.11
cat	**0.82**
sat	0.07
Total	**1.00**

Visualisation: Attention Weights



Key Takeaways

- Softmax converts raw scores into a probability distribution (attention weights).
- The weights sum to 1.00.
- Tokens with higher weights have more influence on the final output.
- In this example, “cat” receives the highest weight (0.82), so it will contribute the most to the resulting representation.

Higher weight → More influence

What Happens to the Values?

Attention weights determine how much of each Value is used to create the output.

Tokens	Attention Weights		Value V (vector representation)		Weighted Values		Weighted Combination (output vector)
The	0.11	×	$V_1 = [\ 0.2 \quad -0.1 \quad 0.4\ ]$	→	$0.11 \times V_1 = [\ 0.022 \quad -0.011 \quad 0.044\ ]$	=	$[\ 0.814 \quad 0.455 \quad -0.195\ ]$
cat	0.82	×	$V_2 = [\ 1.0 \quad 0.5 \quad -0.3\ ]$	→	$0.82 \times V_2 = [\ 0.820 \quad 0.410 \quad -0.246\ ]$		
sat	0.07	×	$V_3 = [\ -0.4 \quad 0.8 \quad 0.1\ ]$	→	$0.07 \times V_3 = [\ -0.028 \quad 0.056 \quad 0.007\ ]$		Context-aware representation

How It Works

1. Each token has a Value (V).
2. Multiply each Value by its attention weight.
3. Sum the weighted Values.
4. The result is a new representation for the token.

Key Idea

Relevant Values contribute more to the output.

Higher attention weight
→ Larger influence
on the final representation.

Takeaway

Attention doesn't copy a single token — it combines information from all tokens, using the Values and their attention weights.

The output is a weighted combination of all Values.

Q, K and V Together

- Query, Key and Value — Different Roles
- Query (Q)
- What am I looking for?
- Key (K)
- What information do I represent?
- Value (V)
- What information should I provide?
- Attention
- Query + Keys → Weights → Combine Values

Activity: Q, K or V?

- 🎯 **Activity — Match the Role**
- Which component answers each question?
- **A.**
- **“What am I looking for?”**
- **B.**
- **“How relevant am I to the Query?”**
- **C.**
- **“What information do I contribute?”**
- Choose:
- **Query — Key — Value**

What Is Self-Attention?

- **Self-Attention**
- In **self-attention**, tokens in a sequence attend to **other tokens in the same sequence**.
- Example:
- **“The animal didn't cross the street because it was tired.”**
- To understand **“it”**, the model can attend strongly to:
- **“animal”**
- **Key Idea**
- **Each token can use information from other relevant tokens.**

How Self-Attention Works

Self-attention allows each token to use information from other tokens in the same sequence.

Input sequence The | cat | sat | on | the | mat → For each token, we create Query, Key and Value vectors and attend to all tokens in the sequence.

1 Create Q, K and V

For each input token, create Query (Q), Key (K) and Value (V) vectors using learned linear projections.

Tokens	Q	K	V
The	■	■	■
cat	■	■	■
sat	■	■	■
on	■	■	■
the	■	■	■
mat	■	■	■

Each token has a Q, K and V vector (of dimension d).

2 Compare Query with all Keys

For a token (e.g., "cat"), compute the similarity between its Query and the Keys of all tokens (dot products).

Query: "cat" ●●●●●

Token	Key	Score (Q · K)
The	→ ●●●●●	0.12
cat	→ ●●●●●	0.95
sat	→ ●●●●●	0.28
on	→ ●●●●●	0.10
the	→ ●●●●●	0.15
mat	→ ●●●●●	0.32

3 Calculate attention weights

Scale the scores and apply softmax to get attention weights (probabilities).

Token	Attention Weight
The	0.11
cat	0.82
sat	0.07
on	0.03
the	0.05
mat	0.02

Weights sum to 1.00

4 Combine the Values

Multiply each Value vector by its attention weight and sum them up.

Token	Weight	Value (V)	Weighted Value
The	0.11	●●●●●	●●●●●
cat	0.82	●●●●●	●●●●●
sat	0.07	●●●●●	●●●●●
on	0.03	●●●●●	●●●●●
the	0.05	●●●●●	●●●●●
mat	0.02	●●●●●	●●●●●

Σ

Weighted sum of all values = Context vector for "cat"

5 Context-Aware Representation

The result is a new representation for the token that incorporates information from other relevant tokens.

Output representation (for "cat")

●●●●●

↓

Context-aware representation

Contains information from other tokens (e.g., "The", "sat", "mat")

Key Idea: Each token can attend to other tokens in the same sequence. Relevant tokens contribute more to the new representation.

"Self-attention lets every token see the whole context."

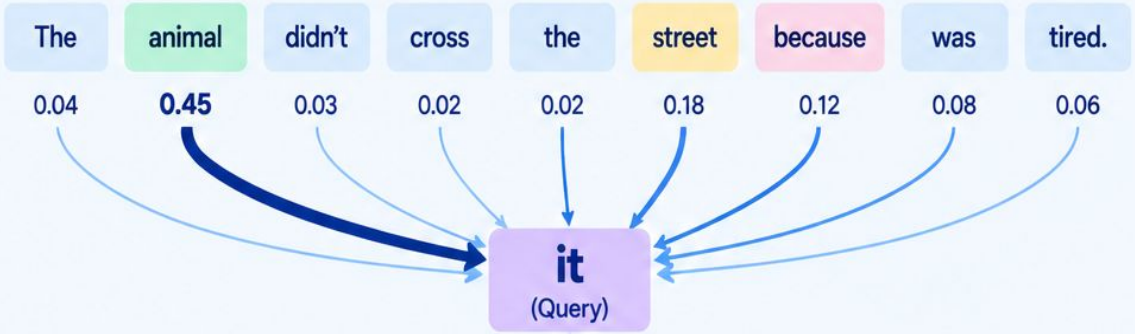
Self-Attention Example

Which Words Should "It" Attend To?

The	animal	didn't	cross	the	street	because	it	was	tired.
1	2	3	4	5	6	7	8	9	10

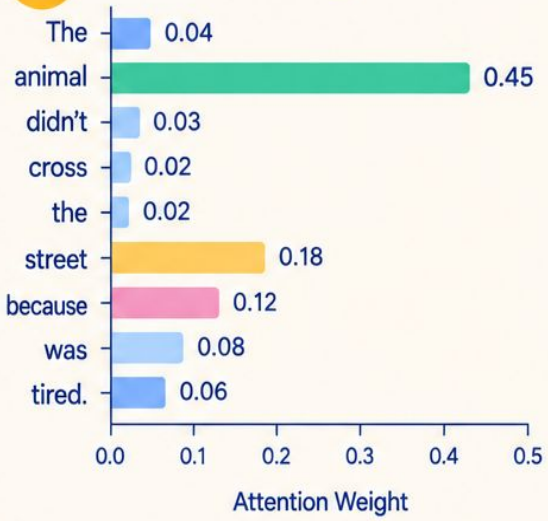
1 Self-Attention from "it"

The token "it" attends to all other tokens and assigns different attention weights based on relevance.



"it" attends to relevant tokens in the same sequence.

2 Attention Weights for "it"



3 Result

The representation of "it" incorporates relevant contextual information, especially from **"animal"** (and to some extent "street" and "because").

"It" now has a context-aware representation that helps the model understand what "it" refers to.

Key Idea

Tokens that are more relevant receive higher attention weights and contribute more to the final representation.

Activity — Find the Relationship

- Consider:
- **“Sara put the book on the table because she needed it later.”**
- **For the token “she”**
- Which token should be highly relevant?
- **A. Sara**
  B. book
  C. table
- **For the token “it”**
- Which token could be highly relevant?
- **A. Sara**
  B. book
  C. because

Why Self-Attention Is Powerful

- **Advantages of Self-Attention**
- **Direct Relationships**
- Tokens can directly interact with other tokens.
- **Long-Range Connections**
- Distant tokens can influence each other without passing through every intermediate recurrent step.
- **Parallel Processing**
- Many token representations can be computed in parallel during training.
- **Context-Aware Representations**
- A token's representation can change depending on surrounding context.

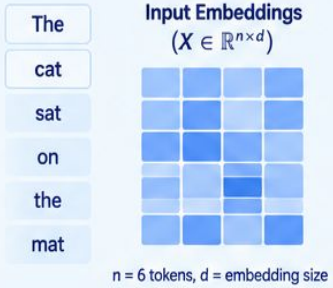
Attention: Complete Picture

From Input to Context-Aware Representation

"Attention lets each token use information from all tokens in the sequence."

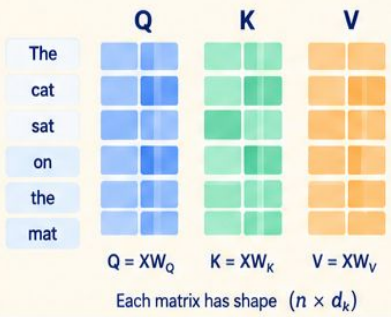
1 Input Tokens

A sequence of tokens is converted into vector representations (embeddings).



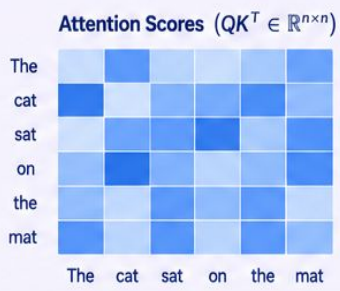
2 Create Q, K, V

Each token is projected into Query, Key and Value vectors using learned linear transformations.



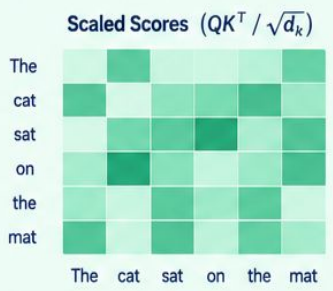
3 Q × K^T

Compute attention scores by comparing each Query with all Keys (dot products).



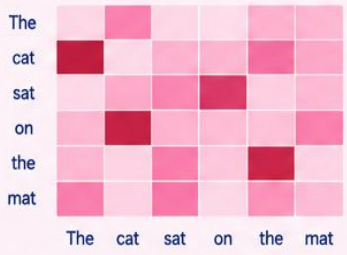
4 Scale by $\sqrt{d_k}$

Divide the scores by the square root of the key dimension to keep values in a stable range.



5 Softmax

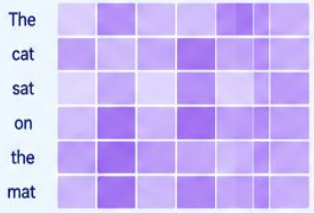
Apply softmax to convert scores into attention weights (probabilities).



Each row sums to 1.0

6 Weighted Values

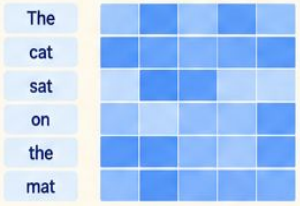
Multiply the attention weights with the Value vectors.



Each token becomes a weighted combination of all Value vectors.

7 Context-Aware Representations

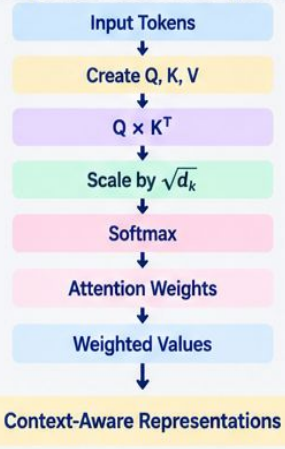
The output is a new representation for each token that incorporates information from other tokens.



$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$(Z \in \mathbb{R}^{n \times d_v})$

Complete Attention Pipeline



Key Idea: Attention allows each token to look at all other tokens and combine their information, creating a context-aware representation.

"Same token, different context → different representation."

Lec11

Transformer Components

- We now understand **self-attention**.
- But a Transformer contains more than one attention operation.
- Next we will study:
- **Multi-Head Attention**
- Feed-Forward Networks
- Residual Connections
- Normalization
- Positional Information

Transformer Components

- *Building Blocks of the Transformer*
- Multi-Head Attention • Feed-Forward Networks • Residual Connections • Normalization • Positional Information

Learning Objectives

- By the end of this lecture, you should be able to:
- Explain **multi-head attention**
- Describe the role of **feed-forward networks**
- Explain **residual connections**
- Understand why **normalization** is used
- Explain why Transformers need **positional information**

- From Self-Attention to Transformers
- Self-Attention
  ↓
  Multi-Head Attention
  ↓
  Feed-Forward Network
  ↓
  Residual Connections + Normalization
  ↓
  Transformer Representation
- + Positional Information

- **Multi-Head Attention**
- **Why Multiple Attention Heads?**
- Instead of performing attention once:
- **Input → Attention → Output**
- Perform attention multiple times:
- **Head 1 → Relationship A**
  Head 2 → Relationship B
  Head 3 → Relationship C
- Then:
- **Combine Heads → Output**

Activity: Why Multiple Heads?

- Sentence:
- **“The student submitted the assignment because she finished it.”**
- What relationships might different attention heads capture?
- **she ↔ ?**
- **it ↔ ?**
- Other relationships?

Feed-Forward Network (FFN)

- After attention, each token representation passes through a small neural network.
- **Attention Output**
  ↓
  Linear Layer
  ↓
  Activation
  ↓
  Linear Layer
  ↓
  Output
- **Purpose**
- **Transform and refine each token representation**

Residual Connection

- Instead of using only the new output:
- **Input → Component → Output**
- we preserve the original information:
- **Input + Component Output**
- **Key Idea**
- **Keep original information while learning new information**

Normalization?

- Normalization helps keep internal representations well behaved during training.
- **Input**
  ↓
  Attention / FFN
  ↓
  Residual Connection
  ↓
  Normalization
- **Benefits**
- Supports stable training
- Helps optimization
- Used throughout Transformer blocks

Activity: Match the Component

Match each function with the correct Transformer component.

"Different components, different roles, a stronger Transformer."

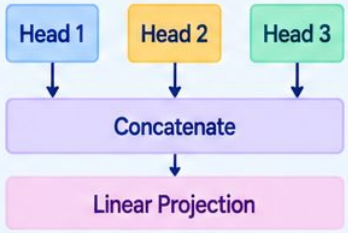
A Captures multiple relationships. → ?

B Transforms each token representation. → ?

C Preserves information through a shortcut. → ?

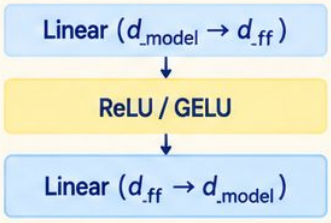
D Helps stabilize internal representations. → ?

Multi-Head Attention



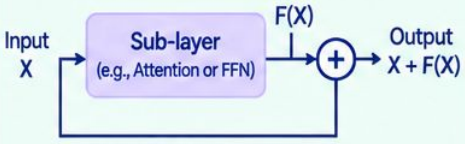
Attends to the input using multiple heads to capture different relationships.

Feed-Forward Network (FFN)



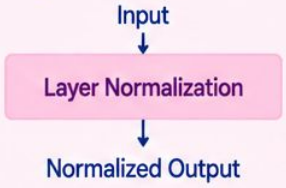
Transforms each token representation with a non-linear function.

Residual Connection



Preserves information through a shortcut connection.

Normalization (Layer Normalization)



Helps stabilize internal representations and improves training.

Your Task:
Match each function (A–D) with the correct component from the right.

Tip: Think about the main purpose of each component in a Transformer.

The Position Problem

- Consider:
- **Dog bites man**
- and
- **Man bites dog**
- They contain similar tokens, but:
- **Order changes meaning.**
- **Question**
- How does a Transformer know token order?

Positional Information

Position information is added or otherwise incorporated into token representations.

This helps the model distinguish the same token at different positions (e.g., Token 1 ≠ Token 5).

Token Representation
(from embedding layer)

$x_t \in \mathbb{R}^d$

+

Position Information
(positional encoding)

$p_t \in \mathbb{R}^d$

↓

Position-Aware Representation

$z_t = x_t + p_t \in \mathbb{R}^d$

Example: Same Token at Different Positions

Position 1	Position 5
cat (Token 1)	**cat** (Token 5)
Token representation x_1	Token representation x_5
0.2 0.8 -0.1 0.4	0.2 0.8 -0.1 0.4
+	+
Position encoding p_1	Position encoding p_5
0.1 0.0 0.1 0.0	-0.1 0.2 0.0 0.1
↓	↓
Position-aware representation z_1	Position-aware representation z_5
0.3 0.8 0.0 0.4	0.1 1.0 -0.1 0.5

Same word, different positions → different representations!

Why It Is Needed?

1. Self-attention does not inherently know the order of tokens.
2. Without position information, the model would treat a sentence like a bag of words.
3. Position encodings give each token a sense of its position in the sequence.

Example: Order Matters

The	cat	sat	on	the	mat.
1	2	3	4	5	6

On	the	mat	sat	the	cat.
1	2	3	4	5	6

Same words, different meaning because of position!

Key Takeaway: Position information is added to token representations so that the model can understand the order of tokens. This helps distinguish Token 1 ≠ Token 5.

"Position gives meaning to the sequence."

Activity: Does Order Matter?

The same words can have different meanings depending on their order.

Sentence 1

"The dog chased the cat."



The	dog	chased	the	cat.
1	2	3	4	5

Sentence 2

"The cat chased the dog."



The	cat	chased	the	dog.
1	2	3	4	5

1 Same words?

Mostly yes.

Both sentences use the same set of words (The, dog, chased, the, cat).

2 Same meaning?

No.

The first sentence means the dog chased the cat, while the second means the cat chased the dog.

3 Why?

Position and order matter.

The meaning of a sentence depends on the order of the words, so position information is important.

Key Takeaway: Transformers use positional information so that the model can distinguish tokens at different positions in the sequence.

"Same words, different meaning — because of order!"

Activity Answer — Does Order Matter?

- **Sentence 1**
- **“The dog chased the cat.”**
- **Dog = doing the chasing**
  Cat = being chased
- **Sentence 2**
- **“The cat chased the dog.”**
- **Cat = doing the chasing**
  Dog = being chased
- ✅ **Answer**
- The words are mostly the same, but **changing their positions changes their roles and the sentence meaning**.
- **Key Idea**
- **Word order carries information → Transformers need positional information.**

Putting the Components Together

Transformer Block — Simplified

Each component plays a specific role, and together they enable powerful sequence modelling.

1 Input + Position

Token embeddings are combined with positional information to give each token a sense of its position in the sequence.

2 Multi-Head Attention

Allows each token to attend to other tokens in the sequence using multiple attention heads to capture different relationships.

3 Residual + Normalization

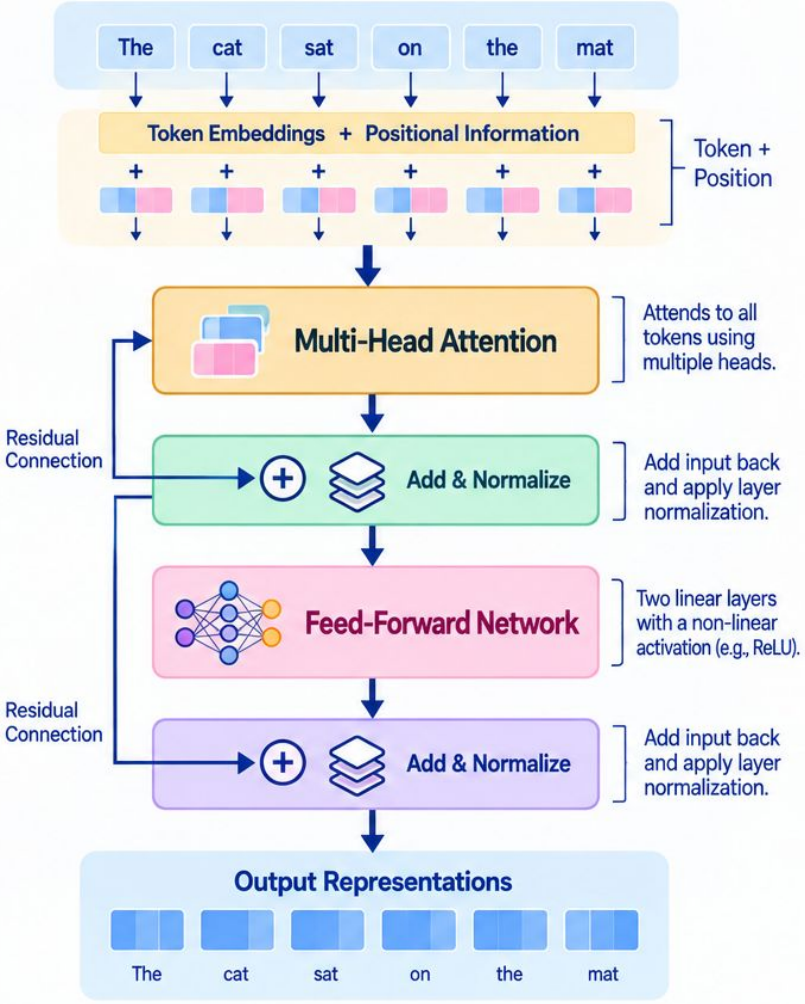
A residual connection adds the input back to the output, and normalization stabilizes training.

4 Feed-Forward Network

A position-wise neural network that transforms each token representation independently (with non-linear activation).

5 Residual + Normalization

Again, a residual connection and normalization are applied.



Transformer Block (Simplified)

Input + Position
↓
Multi-Head Attention
↓
Residual + Normalization
↓
Feed-Forward Network
↓
Residual + Normalization
↓
Output Representations

Key Takeaway

These components work together to create context-aware, position-aware representations that are more powerful than traditional sequential models.

Lec12

Transformer Architectures

- From Transformer Blocks to Complete Models

Learning Objectives

- By the end of this lecture, you should be able to:
- Distinguish the three main Transformer architectures
- Explain **encoder-only** models
- Explain **decoder-only** models
- Explain **encoder-decoder** models
- Describe **causal attention**

Three Main Architectures

- **Encoder-Only**
- **Input → Encoder → Representation**
- Best suited to understanding-oriented tasks.
- **Decoder-Only**
- **Prompt → Decoder → Generated Tokens**
- Common for generative language modelling.
- **Encoder–Decoder**
- **Input → Encoder → Decoder → Output**
- Useful for sequence-to-sequence tasks.

Encoder-Only Architecture

Understand the input. Encode it. Use the representations for a specific task.

"Encoder-only models read the entire input and learn rich contextual representations."

Input Tokens

A sequence of tokens (e.g., a sentence).

[CLS] | The | movie | was | great | [SEP]

Transformer Encoder

A stack of Transformer encoder layers (self-attention and feed-forward networks).

- Transformer Encoder Layer 1
- Transformer Encoder Layer 2
- ⋮
- Transformer Encoder Layer N

N × Encoder Layers

Contextual Representations

Context-aware vector representations for each token (output from the last encoder layer).

[CLS] | The | movie | was | great | [SEP]

Task Output

Use the contextual representations for a specific downstream task.

Task-Specific Layer (e.g., classification head) → **Output** (e.g., class label, sentiment, entities, etc.)

Common Tasks

- **Text classification** (e.g., spam detection, topic classification)
- **Sentiment analysis** (e.g., positive, negative, neutral)
- **Named entity recognition** (e.g., person, organization, location)
- **Representation / embedding tasks** (e.g., sentence embeddings, similarity)

Example Family

BERT-style models

- BERT
- RoBERTa
- ALBERT
- DistilBERT
- and other encoder-only models

Key Takeaway: Encoder-only models read the entire input using self-attention to produce contextual representations, which are then used for various NLP tasks.

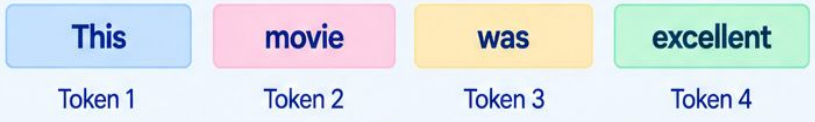
"Same text, richer meaning through context."

Encoder-Only: Key Idea

The encoder reads the input and produces context-aware representations for the tokens.

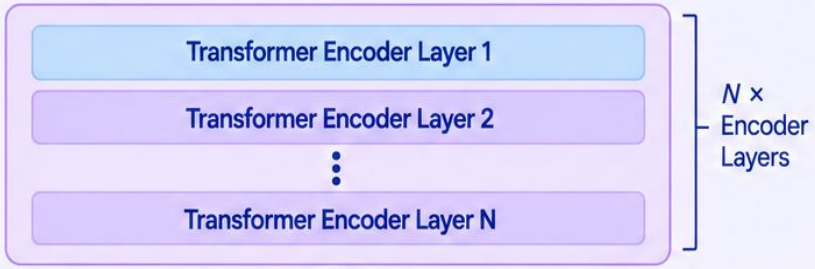
Input

A sequence of tokens (e.g., a sentence).



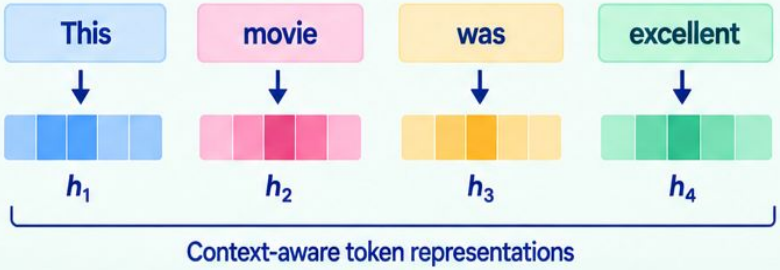
Encoder

A stack of Transformer encoder layers.



Contextual Representations

The encoder considers the input context and produces a context-aware representation for each token.



Example

"This movie was excellent."

The encoder considers the input context and produces representations for the tokens.

Key Idea

Read and represent the input.

The encoder reads the entire input using self-attention and creates context-rich representations for each token.

Result

Each token now has a representation that captures the meaning of the token in the context of the whole input.

"The same word can have different representations depending on its context."

Activity — Encoder or Not?

- Which task is most naturally suited to an encoder-only model?
- **A. Sentiment classification**
- **B. Generate a story one token at a time**
- **C. Translate English into French**
- **Your Answer?**

Activity Answer

- ✅ **Answer**
- **A. Sentiment Classification**
- **Input**
- “The product is excellent.”
- ↓
- **Encoder**
- ↓
- **Positive / Negative**
- **Key Idea**
- **Encoder-only models are well suited to input understanding and representation tasks.**

Decoder-Only Architecture

Generate text one token at a time, from left to right.

How It Works

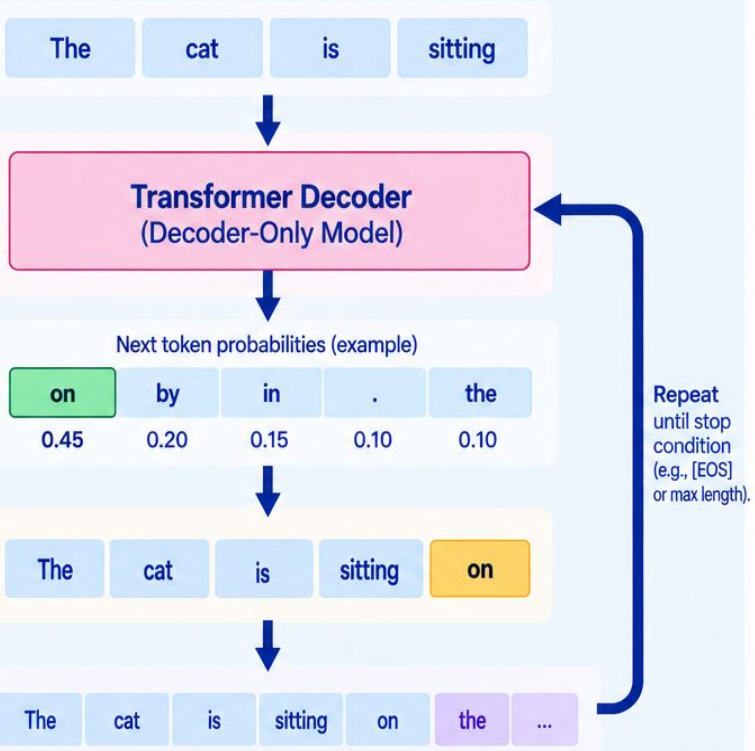
1 Prompt
A sequence of tokens as input (context).

2 Transformer Decoder
Processes the input using masked self-attention (causal attention) so it only sees past tokens.

3 Predict Next Token
Predicts the probability distribution for the next token.

4 Add Token
Select the most likely token (e.g., "on") and add it to the sequence.

5 Predict Again
Repeat the process to predict the next token.



Common Tasks

- **Text generation** (e.g., stories, articles, summaries)
- **Dialogue** (e.g., chatbots, assistants)
- **Completion** (e.g., autocomplete)
- **Code generation** (e.g., program synthesis)

Example

Prompt: The cat is sitting
Model: on
Model: the
Model: mat
Model: .

Generated output:

The cat is sitting on the mat.

Key Takeaway: Decoder-only models generate text autoregressively, predicting one token at a time using only the previous tokens.

"Generate, one token at a time."

Encoder-Only Architecture

Understand the input. Encode it. Use the representations for a specific task.

"Encoder-only models read the entire input and learn rich contextual representations."

Input Tokens

A sequence of tokens (e.g., a sentence).

[CLS] | The | movie | was | great | [SEP]

Transformer Encoder

A stack of Transformer encoder layers (self-attention and feed-forward networks).

- Transformer Encoder Layer 1
- Transformer Encoder Layer 2
- ⋮
- Transformer Encoder Layer N

N × Encoder Layers

Contextual Representations

Context-aware vector representations for each token (output from the last encoder layer).

[CLS] | The | movie | was | great | [SEP]

Task Output

Use the contextual representations for a specific downstream task.

Task-Specific Layer (e.g., classification head) → **Output** (e.g., class label, sentiment, entities, etc.)

Common Tasks

- **Text classification** (e.g., spam detection, topic classification)
- **Sentiment analysis** (e.g., positive, negative, neutral)
- **Named entity recognition** (e.g., person, organization, location)
- **Representation / embedding tasks** (e.g., sentence embeddings, similarity)

Example Family

BERT-style models

- BERT
- RoBERTa
- ALBERT
- DistilBERT
- and other encoder-only models

Key Takeaway: Encoder-only models read the entire input using self-attention to produce contextual representations, which are then used for various NLP tasks.

"Same text, richer meaning through context."

Encoder–Decoder Models

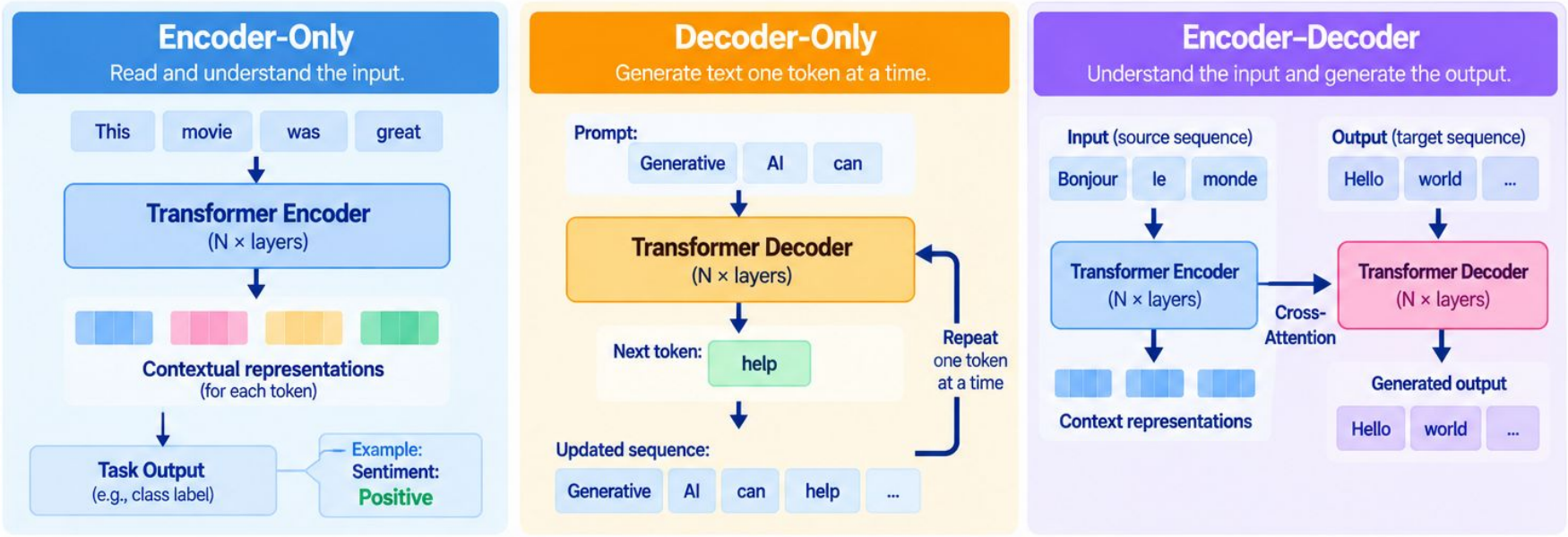
- **Encoder–Decoder Models**
- Two major parts:
- **Encoder**
- **Reads and represents the input**
- ↓
- **Decoder**
- **Generates the output**
- **Pipeline**
- **Input → Encoder → Representation → Decoder → Output**

Encoder–Decoder Example

- **Example: Translation**
- **Input**
- **“Good morning”**
- ↓
- **Encoder**
- Creates input representations
- ↓
- **Decoder**
- Generates:
- **“Bonjour”**
- **Other Applications**
- Translation
- Summarization
- Sequence-to-sequence generation

Compare the Three: Transformer Architectures

Same building blocks (transformer layers), different setups, different goals.



Architecture	Main Purpose	Example Task
Encoder-Only	Represent / understand	Classification
Decoder-Only	Generate	Text generation
Encoder–Decoder	Transform sequence	Translation

Easy Way to Remember

Think about what each model does.

Encoder → Understand
Reads and understands the input.

Decoder → Generate
Generates text one token at a time.

Encoder–Decoder → Understand + Generate
Understands the input and generates the output.

Activity: Choose the Architecture

- 🧩 **Activity — Which Architecture?**
- Match each task:
- **1. Sentiment Analysis**
- **2. Story Generation**
- **3. English → French Translation**
- Choose:
- **A. Encoder-Only**
- **B. Decoder-Only**
- **C. Encoder–Decoder**

Activity Answer

- ✅ Answer
- Sentiment Analysis
- Encoder-Only
- Story Generation
- Decoder-Only
- Translation
- Encoder–Decoder
- Remember
- Understand → Encoder
- Generate → Decoder
- Input → Output transformation → Encoder–Decoder

Transformer Inference

- **Inference = using a trained model to produce outputs for new inputs**
- Example:
- **User Prompt**
- “Explain artificial intelligence.”
- ↓
- **Trained Transformer**
- ↓
- **Model Output**
- “Artificial intelligence is…”

When Does Generation Stop?

- **Stopping Generation**
- Generation can stop when:
- A special **end-of-sequence token** is produced
- A maximum token limit is reached
- A system/application stopping condition is met
- **Generation Loop**
- **Predict → Select → Add → Repeat → Stop**

Putting Week 4 Together
From Attention to Transformers

Same building blocks.
Different combinations.
Powerful models.

10

Lecture 10
Attention
Q, K, V → Attention → Self-Attention

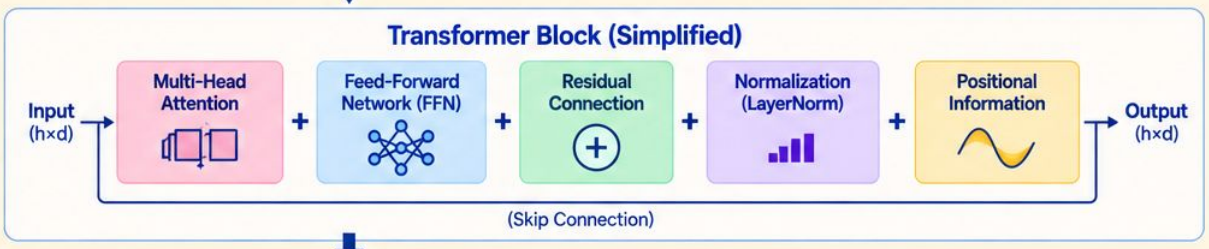
Learn how attention works and how it becomes self-attention.



11

Lecture 11
Components
Multi-Head Attention + FFN + Residual + Normalization + Position

Combine attention with other components to build a Transformer block.



12

Lecture 12
Architectures
Encoder | Decoder | Encoder–Decoder

Use Transformer blocks to build different architectures for different tasks.



Key Takeaway

Attention is the core idea, the components build the Transformer block, and different architectures use these blocks for different tasks.

"From attention to real-world models."

What Is Causal Attention?

- **Causal Attention**
- During next-token prediction:
- **A token can attend to:**
- **Previous tokens + itself**
- **But NOT:**
- **Future tokens**
- Example:
- I → love → Generative → AI
- When processing **Generative**:
- ✅ I
  ✅ love
  ✅ Generative
  ❌ AI

Why Causal Attention?

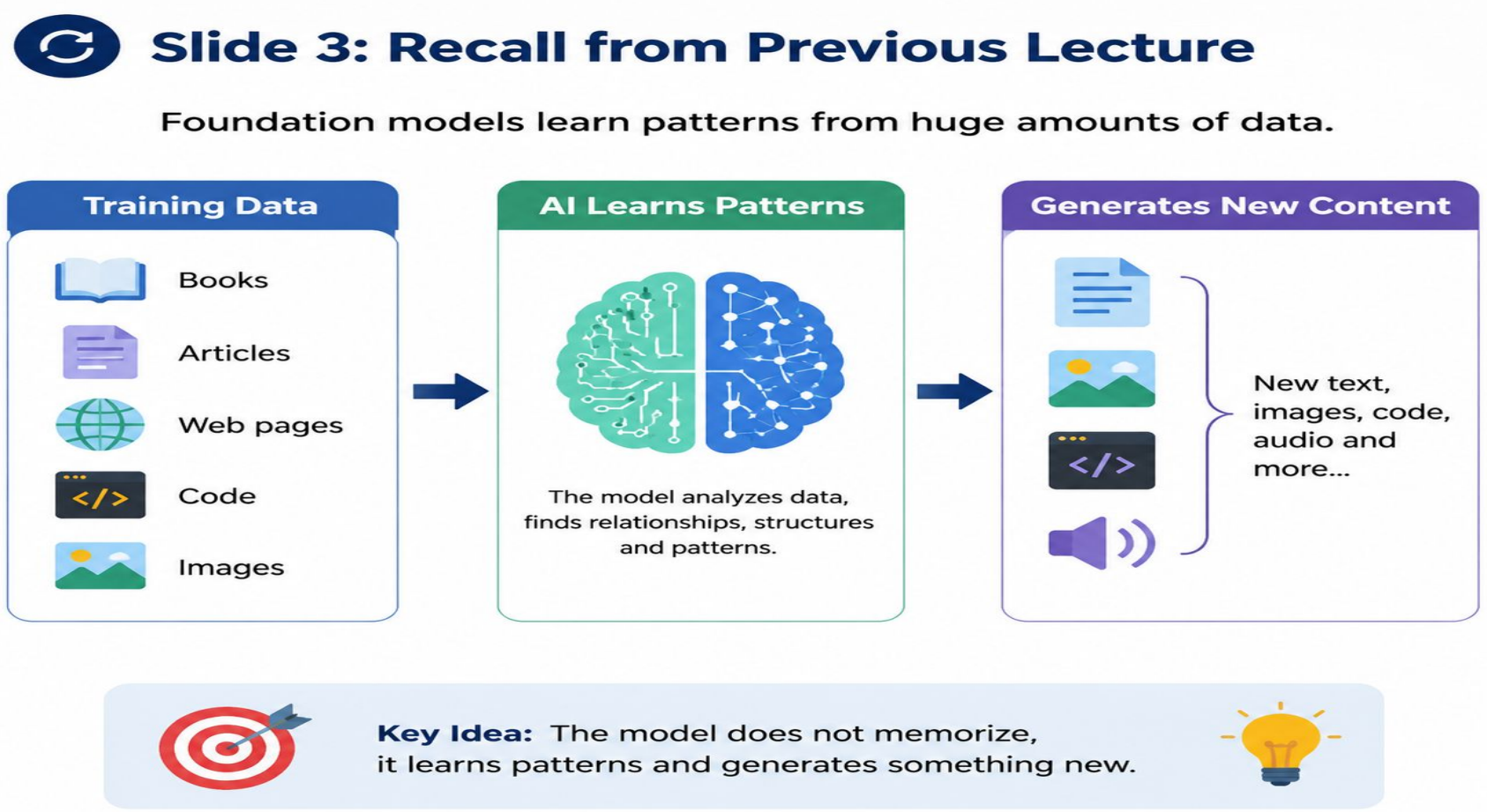
- **Why Hide Future Tokens?**
- Suppose:
- **“I drink coffee every morning.”**
- When predicting:
- **“coffee”**
- The model may use:
- **I | drink**
- But it should NOT see:
- **every | morning**
- **Why?**
- Otherwise, the model could access future information.
- **Key Idea**
- **No looking ahead!**

Activity — Causal Attention

- Sequence:
- **AI can help students learn**
- When processing **“students”**, which tokens can it attend to?
- **A. AI**
- **B. can**
- **C. help**
- **D. students**
- **E. learn**
- **Which one is blocked?**

- **Activity Answer**
- ✅ **Answer**
- When processing **“students”**:
- ✅ AI
  ✅ can
  ✅ help
  ✅ students
  ❌ **learn**
- **Why?**
- **“learn” is a future token.**
- **Causal Attention**
- **Past + Present → Allowed**
- **Future → Blocked**

Why Probability in Generative AI?



How does a model decide what to generate?

- If ChatGPT generates:
- "Artificial intelligence is changing the world..."
- How does the model decide which word comes next?
- Possible next words:
- rapidly
- today
- because
- through

The model does not "know".

It calculates probabilities.

Slide 5: Key Idea

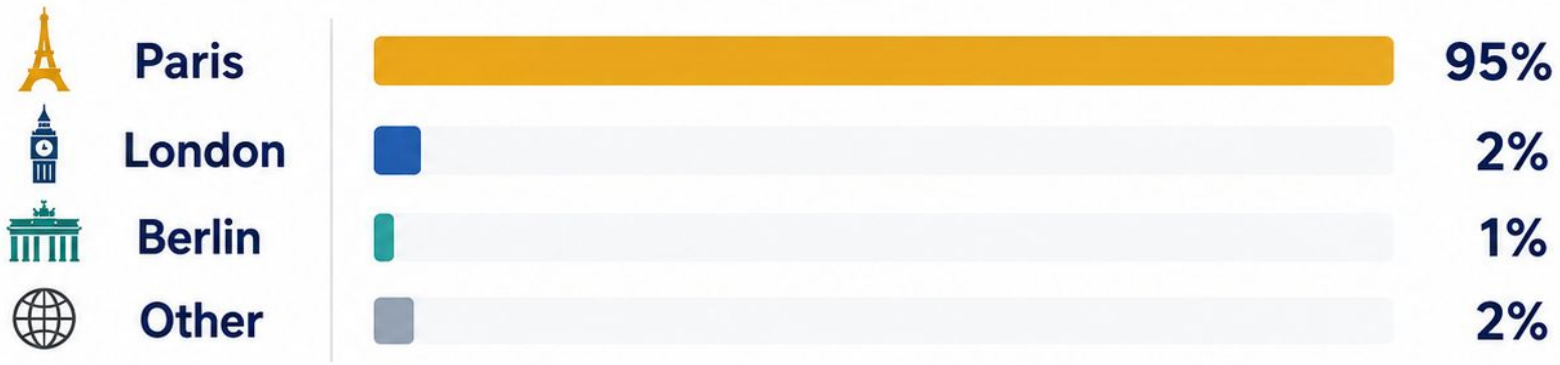
Generative AI is based on:

Learning Probability Distributions

A model learns: *'How likely is each possible output?'*

Example

After: "The capital of France is"



Key Message: The model assigns probabilities to possible outputs and chooses the most likely next token.

What is Probability?

Probability measures the likelihood that an event happens.

- Range:
- $0 \leq$ Probability ≤ 1
- 0 = impossible
- 1 = certain

Examples of Probability

Probability helps us quantify the likelihood of an event happening.

EXAMPLES

Coin

A coin has two possible outcomes: Head or Tail.

HEAD



TAIL



Probability of Head
= 0.5 (50%)

Equal chance of getting Head or Tail.

Dice

A standard dice has 6 possible outcomes: 1, 2, 3, 4, 5, 6.



Probability of rolling 6
= $\frac{1}{6}$ = 0.167 (16.7%)

Only 1 favorable outcome out of 6 possible outcomes.

Weather

Weather forecasts use probability to express uncertainty.

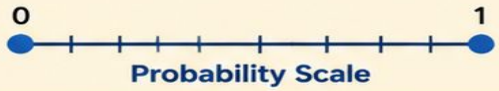


Rain probability
= 70% (0.70)

Means rain is likely, but not certain.

Key Takeaway: Probability values range from 0 to 1.

0 = Impossible to **1 = Certain**



In **Generative AI**, models use **probability** to decide which outputs are more likely and generate new content.

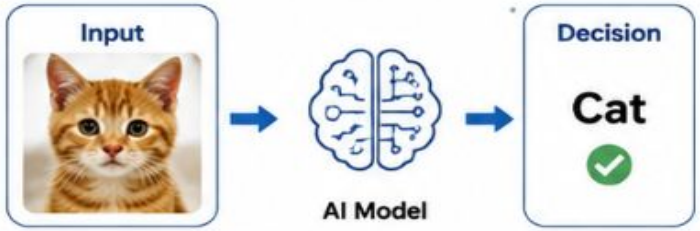
Probability in AI

AI systems use probability to decide or generate outputs.

Traditional AI

Focuses on making a decision or prediction.

Input → Decision



Example

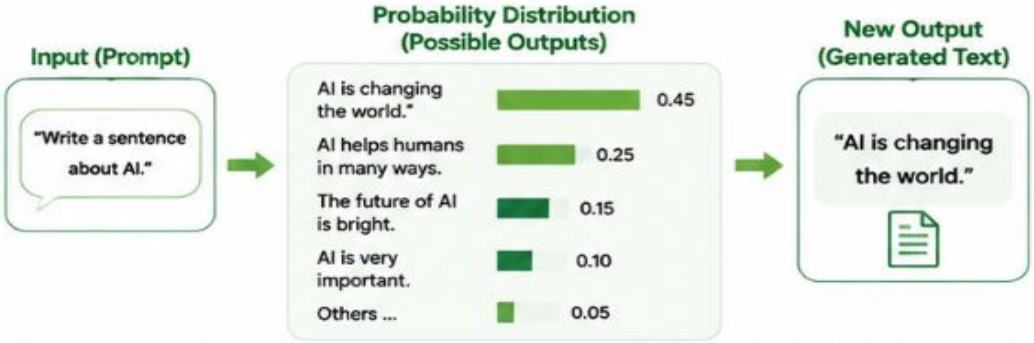


Goal: Find the correct answer / class from given input.

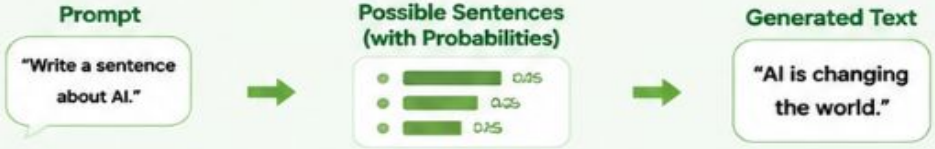
Generative AI

Focuses on generating new content using probabilities.

Input → Probability Distribution → New Output



Example



Goal: Learn the probability of possible outputs and generate new, meaningful content.

Key Difference

Traditional AI predicts the best answer.

Example: Image → Cat

VS

Generative AI generates new content based on probabilities.

Example: Prompt → Possible sentences → Generated text

Activity

Task	Prediction or Generation?
AI detects whether an X-ray shows pneumonia	?
AI writes a patient medical report	?
AI identifies a cat in an image	?
AI creates a new image of a cat	?
AI predicts tomorrow's temperature	?

- X-ray → Prediction
- Medical report → Generation
- Cat detection → Prediction
- New cat image → Generation
- Temperature prediction → Prediction

Activity

- A language model predicts the next word:

- Input:
- "The weather today is“
- "For example:
- sunny → 60%
- rainy → 25%
- cold → 10%
- beautiful → 5%“

- Which word is most likely generated???

Question: A Language Model Predicts the Next Word

Input

"The weather today is"



Language Model

Task

Predict the next word

Possible Outputs and Their Probabilities

Word	Probability
sunny	60%
rainy	25%
cold	10%
beautiful	5%

Key Idea

The model assigns a probability to each possible next word.

The sum of all probabilities = 100%

Explanation:

The language model looks at the input and gives probabilities for all possible next words. The word with the highest probability ("sunny") is the most likely prediction.

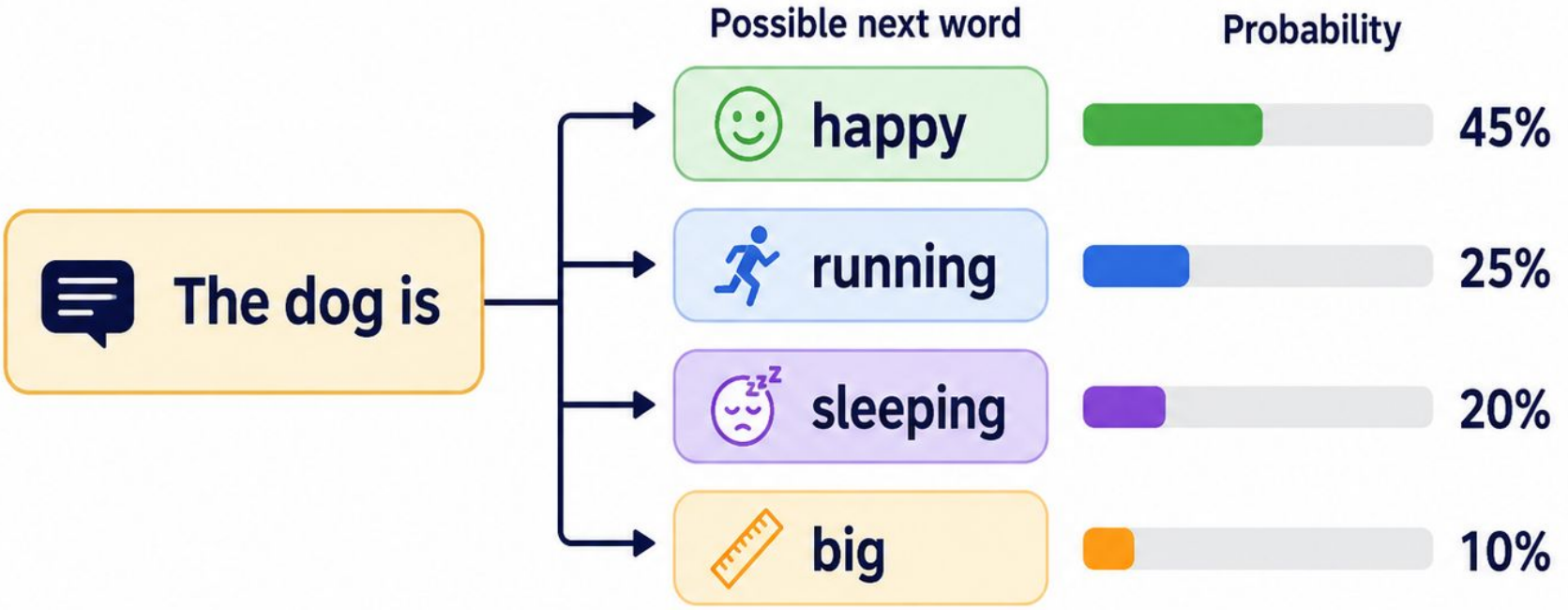
$$\sum 60\% + 25\% + 10\% + 5\% = 100\%$$

Random Variables

- **What is a Random Variable?**
- A variable whose value depends on chance.
- Example:
- Rolling dice:
- Random variable:
- X = Dice result
- Possible values:
- X = {1,2,3,4,5,6}

Random Variables in GenAI

In text generation, the next word is a random variable.



Each possible next word has a probability, and the model selects one based on the learned distribution.

Activity

- Identify the random variable:
- Example:
- Input:
- "I am going to the"
- Possible outputs:
- market
- school
- park
- office

- **What is the random variable in this example?**
- Options:
- A. The sentence
  B. The next word
  C. The AI model
  D. The training data

- **Why?**
- "In this example, the input sentence is fixed:
- **'I am going to the...'**
- The uncertainty is about **what word will come next**."
- "The AI model considers different possible words, such as:
- market
- school
- park
- office
- Each word has a different probability of being selected."
- "Therefore, the **random variable is the next word**, because its value can change depending on chance and probability."
-

- **Why other options are incorrect?**
- ❌ **A. The sentence**
  "The sentence is the input or context provided to the model. It is not uncertain because it is already given."
- ❌ **C. The AI model**
  "The model is the system that calculates probabilities. It does not represent the random outcome."
- ❌ **D. The training data**
  "The training data is used to learn patterns, but it is not the uncertain value being generated."

What is a Probability Distribution?

- A probability distribution shows:
- "How probability is assigned to possible outcomes."
- Example:
- Next word prediction:
- Input:
- "I like"
- Distribution:
- pizza 40%
- music 25%
- coffee 20%
- books 15%

Probability Distribution Example

- Generating an image:
- Prompt:
- "A picture of a dog"
- Possible outputs:
- Golden retriever 50%
- Husky 30%
- Bulldog 20%
- The model samples from this distribution.

Types of Probability Distributions

- Two main types:
- **1. Discrete Distribution**
- Finite possible values
- Example:
- Words
- **2. Continuous Distribution**
- Infinite possible values
- Example:
- Image pixels, audio signals

Types of Probability Distributions

Two main types of probability distributions are used in AI and Generative AI.

1 Discrete Distribution



Finite possible values
The variable can take countable, separate values.

Example: Words

When generating the next word,
the model considers a fixed set of possible words.

Word (Possible Value)	Probability
happy	0.35
running	0.25
sleeping	0.20
big	0.10
other words	0.10



2 Continuous Distribution

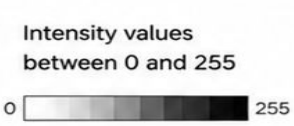


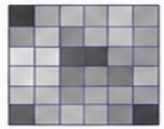
Infinite possible values
The variable can take any value within a range or interval.

Example: Image Pixels, Audio Signals

Pixel intensities or audio amplitudes
can take any real value within a range.

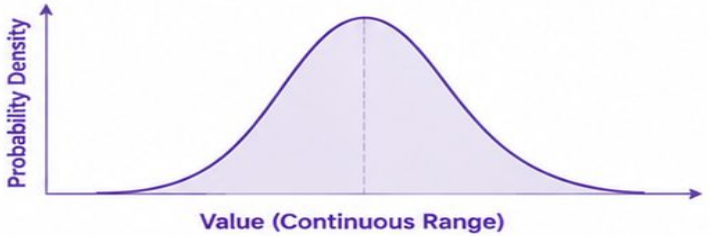
Image pixels: Intensity values between 0 and 255





Audio signals: Amplitude values in a continuous range







Key Takeaway: Discrete distributions deal with countable outcomes (like words), while continuous distributions deal with values in a range (like pixels or audio).

Main Idea of Generative Models

- A generative model tries to learn:
- **P(Data)**
- "The probability distribution of the data."
- Example:
- Images of cats:
- Model learns:
- Shapes
- Colors
- Patterns
- Features

Joint Probability?

- Joint probability means:
- Probability of two events happening together.
- Notation:
- $P(A, B)$
- or
- P(A∩B)

- Events:
- A = It is raining
- B = Person carries umbrella
- Joint probability:
- Probability that:
- "It rains AND person carries umbrella"

Joint Probability in Language Models

1 Sentence

“The cat sits”

2 What the model considers



The language model looks at the entire sentence as a whole and calculates the probability of **all words occurring together**.

3 Joint Probability

P(The, cat, sits)

Probability of the words “The”, “cat”, and “sits” **occurring together in this exact order.**

How it works

Start (beginning) → START

The	×	cat	×	sits	=	P(The, cat, sits)
P(The)		P(cat | The)		P(sits | The, cat)		Joint probability of the full sentence



Joint probability tells us how likely a complete sequence of words is.

Key Takeaway

In language models, joint probability measures the likelihood of all words in a sentence appearing together in the same order.



Language Generation

- A sentence probability:
- $P(w_1, w_2, w_3, \ldots w_n)$
- Example:
- "The dog runs"
- =
- P(The) × P(dog|The) × P(runs|The dog)

Conditional Probability?

- Probability of an event given another event happened.
- Formula:
- $P(A \mid B)$
- Read:
- "Probability of A given B"

- **Conditional Probability in LLMs**
- LLMs generate:
- Next word based on previous words.
- Example:
- "The capital of France is"
- Model calculates:
- $P(\text{word} \mid \text{previouswords})$

Next Token Prediction

1. Input

"The sun rises in the"

The model receives the previous words (context).

2. Model calculates P(next token | previous words)

Possible next tokens (with probabilities)	
morning	80%
sky	10%
east	5%
day	5%

The model assigns a probability to all possible next tokens.

3. Model selects a token

Selected next token:

morning

The token with the highest probability is chosen.

Resulting text after adding the next token:

The sun rises in the **morning**

Maximum Likelihood Estimation

- Goal:
- Find model parameters that make observed data most likely.
- Simple idea:
- Choose the model that best explains the data.

- Two models generate text.
- Model A:
- Probability of training sentences:
- 90%
- Model B:
- Probability:
- 40%
- Which is better?
- Model A.

Sampling

What is Sampling?

Sampling means:

Selecting an output from a probability distribution.

1 Probability Distribution

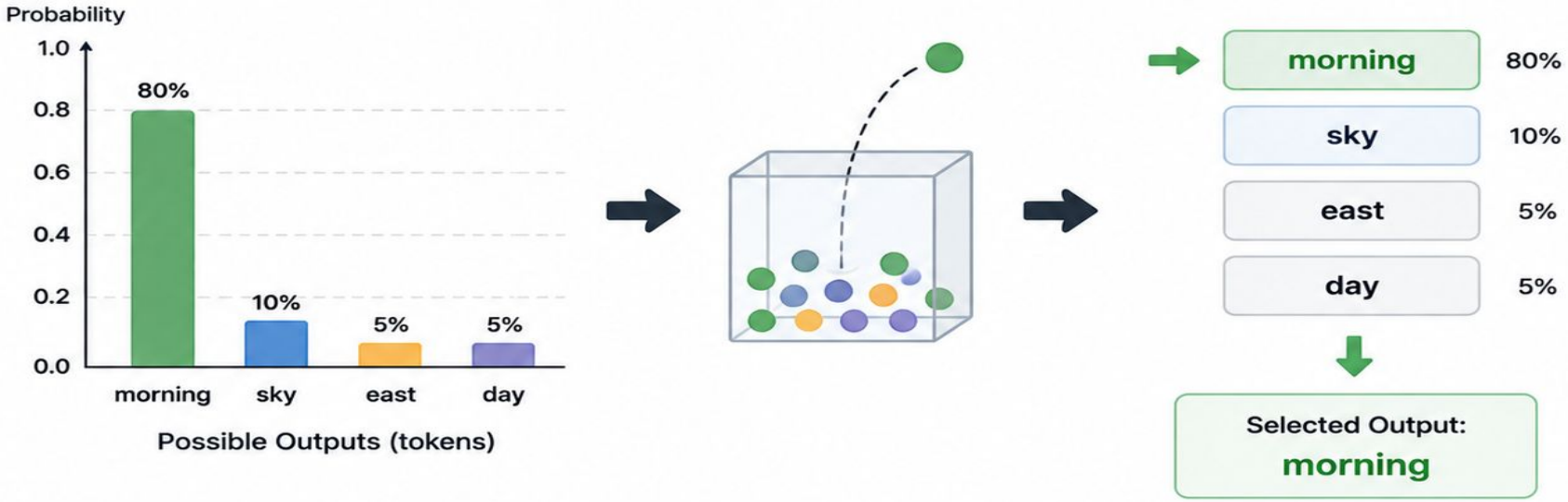
Model assigns probabilities to all possible outputs.

2 Sampling Process

A random draw is made based on these probabilities.

3 Sampled Output

One output is selected as the result.



In Simple Words

Sampling is like picking one ball from a bag, where each ball's chance of being picked is proportional to its probability.

- **Example**
- Distribution:
- Pizza 60%
- Burger 30%
- Pasta 10%
- One sample:
- Pizza
- Another sample:
- Burger

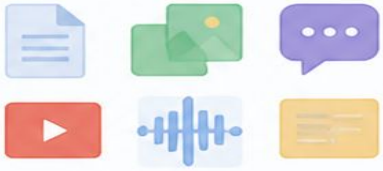
Sampling in Generative AI

- Text generation:
- Probability distribution:
- word1 40%
- word2 30%
- word3 20%
- word4 10%
- Model samples next word.

How Generative Models Work

1. TRAINING

Data



Large amount of training data

Learn probability distribution

Model finds patterns and learns the underlying distribution of the data.

Model



The model now represents the learned probability distribution.

2. GENERATION

Prompt

"Write a short story about a brave knight."

Probability Calculation



Model calculates the probability of possible next tokens (words).

Sampling



One token is randomly selected based on the probabilities.

New Output

"Once upon a time, in a distant land, there lived a brave knight who ..."

The process repeats to generate the complete output.

In Simple Words

The model learns the patterns from data (training). When given a prompt, it predicts what comes next and samples step by step to create new content (generation).

Activity

- Predict next word:
- "The university students are"
- Options:
- A. studying
- B. banana
- C. flying
- Which has highest probability?

Activity

- Generative AI creates images because:
- A. It stores every image
- B. It learns data distribution
- C. It copies images

LEC#5

Autoregressive Generative Models

- **Learning Objectives**
- By the end of the next lecture, students will be able to:
- Understand how autoregressive models generate sequences step-by-step.
- Explain sequence probability and its role in language generation.
- Understand next-token prediction in large language models.
- Explore different decoding strategies used to generate text.

What Are Autoregressive Generative Models?

- Autoregressive models generate new data by predicting the next element based on previously generated elements.
- Key Idea
- Previous tokens → Model → Next token prediction
- Example:
- "The sun rises in the" → morning
- Applications
- Large Language Models (LLMs)
- Text generation
- Machine translation
- Speech generation
- Code generation

Understanding Sequence Generation

Sequence Generation

A sequence is an ordered collection of elements such as words or tokens.

Example:

The → sun → rises → in → the → morning

Autoregressive models generate sequences one step at a time.

Generation Process

1. **Predict first token**
   Model predicts the first token in the sequence.
2. **Use previous tokens as context**
   The generated token(s) become the context for the next prediction.
3. **Predict next token**
   Model predicts the next token based on the current context.
4. **Repeat until completion**
   Repeat the process until an end condition (e.g., end token) is met.

Step-by-Step Visualization

				Next Token Prediction
Step 1	Context (Start)	<START>	Model	The
Step 2	Context	The	Model	sun
Step 3	Context	The sun	Model	rises
Step 4	Context	The sun rises	Model	in
Step 5	Context	The sun rises in	Model	the
Step 6	Context	The sun rises in the	Model	morning
Step 7	Context	The sun rises in the morning	Model	<END>

The model keeps using the previously generated tokens as context to predict the next token, repeating the process until the sequence is complete.

Sequence Probability

Main Concept

The probability of a complete sequence is calculated by multiplying the probability of each token given previous tokens.

The → cat → sleeps → <END>

Formula

$$P(X) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

$P(X)$ = Probability of the complete sequence X

x_i = i-th token in the sequence

$P(x_i|x_1, \dots, x_{i-1})$ = Probability of current token given all previous tokens

Example

Sentence:

The cat sleeps <END>

Probability:

$$P(\text{The}) \times P(\text{cat}|\text{The}) \times P(\text{sleeps}|\text{The, cat}) \times P(\langle\text{END}\rangle|\text{The, cat, sleeps})$$

Step-by-Step Calculation (Example with Hypothetical Probabilities)

1st token	2nd token given previous	3rd token given previous	4th token given previous	Sequence Probability
The	cat	sleeps	<END>	P(X)
P(The) = 0.30	× P(cat|The) = 0.25	× P(sleeps|The, cat) = 0.60	× P(<END>|The, cat, sleeps) = 0.90	= 0.30 × 0.25 × 0.60 × 0.90 = 0.0405

What does it mean?

The model breaks down the sequence into a chain of conditional probabilities.
By multiplying them, it gets the probability of the entire sequence.
Higher probability → the sequence is more likely according to the model.

Key Takeaway

Autoregressive models evaluate how likely a whole sequence is by considering the context at every step.

Next-Token Prediction

Core Mechanism of LLMs:
LLMs generate text by predicting one token at a time.

Example

Input (Context):

"The capital of France is"

Possible Next-Token Predictions

Token	Probability
Paris	95%
London	2%
Rome	1%
...	...

Selected Token

Paris

The token with the highest probability is selected.

Process Flow

1. **Context**
   Current generated text is used as context.
2. **Probability Calculation**
   Model calculates probability distribution over next possible tokens.
3. **Token Selection**
   Select the token with highest probability (or using a sampling strategy).
4. **Continue Generation**
   Add the selected token to the context and repeat the process.

Step-by-Step Generation Example

Step 1	Step 2	Step 3	Step 4	Step 5
Context: The capital of France is	Context: The capital of France is Paris	Context: The capital of France is Paris the	Context: The capital of France is Paris the capital	Context: The capital of France is Paris the capital city
Predicted Next Token: Paris	Predicted Next Token: the	Predicted Next Token: capital	Predicted Next Token: city	Predicted Next Token: of
(Probability: 95%)	(Probability: 72%)	(Probability: 68%)	(Probability: 74%)	(Probability: 71%)

... and so on

Key Takeaway:
LLMs work by repeatedly predicting the next token based on the current context, forming text one token at a time.

Activity

- Think
- **Prompt:**
- "Artificial intelligence is changing the"

- **What is the most likely next word?**
- Like an AI Mode
- Possible options:
- Token
- Probability
- world
- technology
- future

Possible options:

Token	Probability
world	50%
technology	30%
future	20%

Autoregressive Decoding

- Autoregressive decoding is the process of generating a sequence by repeatedly predicting and adding the next token.
- **Example**
- Prompt:
- "The sky is"
- Step 1:
- "The sky is blue"
- Step 2:
- "The sky is blue and"
- Step 3:
- "The sky is blue and clear"
- **Key Point**
- Each generated token becomes part of the context for predicting the next token.

Activity 2: Build a Sentence

- **Title: "Generate Like an LLM"**
- Start with:
- **Prompt:**
- "The student opened the"
- Students choose the next token:
- Step 1:
- book
- laptop
- door
- Assume selected:
- **book**
- New sequence:
- "The student opened the book"
- Step 2:
- Choose next token:
- and
- because
- yesterday
- Continue for 3–4 steps.
- **Learning Point:**
  Each new token becomes the context for predicting the next token.

ACTIVITY ANSWER: "GENERATE LIKE AN LLM"

PROMPT:

"The student opened the"

STEP 1

Choose next token

OPTIONS:

- book ✓
- laptop
- door

SELECTED TOKEN: book

NEW SEQUENCE:
"The student opened the book"

STEP 2

Choose next token

OPTIONS:

- and ✓
- because
- yesterday

SELECTED TOKEN: and

NEW SEQUENCE:
"The student opened the book and"

STEP 3

Choose next token

OPTIONS:

- started ✓
- closed
- forgot

SELECTED TOKEN: started

NEW SEQUENCE:
"The student opened the book and started"

STEP 4

Choose next token

OPTIONS:

- reading ✓
- running
- sleeping

SELECTED TOKEN: reading

NEW SEQUENCE:
"The student opened the book and started reading."

FINAL GENERATED SENTENCE:

"The student opened the book and started reading."

LEARNING POINT:

Each new token becomes the context for predicting the next token.
This is how autoregressive generation works!

Context (previous tokens) → Predict Next Token → Add to Context & Repeat

Decoding Strategies

- **Why Decoding Strategies?**
- Different strategies control how the model selects the next token.
- **Common Strategies**
- **1. Greedy Decoding**
- Always selects the highest probability token.
- Produces predictable outputs.
- Example:
- 80% token selected every time.

2. Beam Search

- Explores multiple possible sequences.
- Selects the sequence with the highest overall probability.
- Used in:
- Translation
- Text generation

DECODING STRATEGIES

Why Decoding Strategies?
Different strategies control how the model selects the next token.

1. GREEDY DECODING

- Always selects the highest probability token.
- Produces predictable outputs.

Example

Prompt: "The sky is"

At each step, the highest probability token is selected.

80% token selected every time.



Prompt: "The sky is"

Step 1		Step 2		Step 3		Step 4
The sky is	→	The sky is blue	→	The sky is blue and	→	The sky is blue and clear
Next Token	Prob.	Next Token	Prob.	Next Token	Prob.	End of sequence
blue	80%	and	70%	clear	75%	
very	10%	but	15%	bright	10%	
really	5%	with	10%	calm	10%	
...	...	...	...	...	...	

Always pick the highest probability token at each step

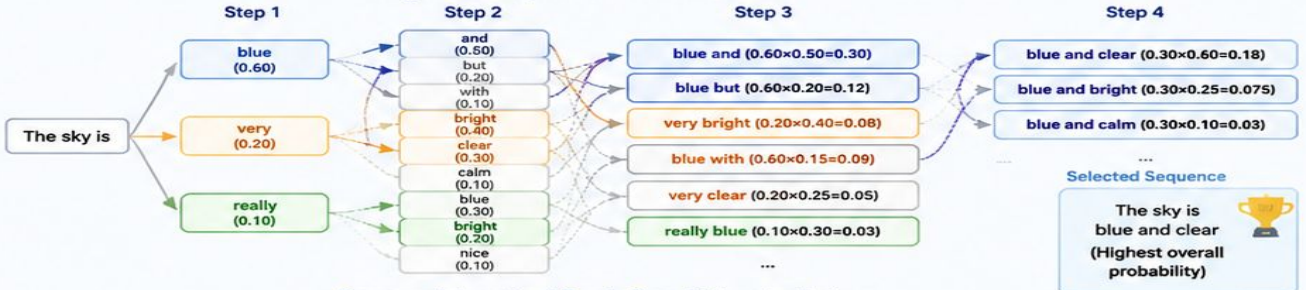
2. BEAM SEARCH

- Explores multiple possible sequences.
- Selects the sequence with the highest overall probability.

Used in

- Translation
- Text Generation

Prompt: "The sky is" | **Beam width = 3**



Beam search keeps top-k (here k=3) candidates at each step

3. SAMPLING-BASED DECODING

- Selects tokens based on probability distribution.
- Produces diverse outputs.

Example

Prompt: "Good"

Token	Probability
morning	80%
sky	10%
east	5%
others	5%

How it works (one possible run)



Different runs can produce different but plausible outputs

Comparison of Decoding Strategies

Strategy	How it works	Pros	Cons	Best used for
Greedy Decoding	Always picks the highest probability token.	Fast, simple, predictable	May miss better long-term sequences	When speed and consistency are important
Beam Search	Explores multiple sequences and picks the best overall.	Better quality, more accurate	Slower, more computational cost	Translation, summarization, high-quality generation
Sampling-Based Decoding	Samples tokens based on probability distribution.	More diverse, creative outputs	May produce less predictable results	Creative writing, chatbots, exploratory generation

Greedy vs Sampling

- Prompt:
- "The robot walked into the"
- **Greedy Decoding Output:**
- "The robot walked into the room."
- **Sampling Output:**
- "The robot walked into the mysterious forest."

Complete Workflow of Autoregressive Generation

From Prompt to Output

1. Prompt

User provides the initial input or instruction.

Example:
"The sky is"

2. Context Representation

The model converts the prompt and previous tokens into a context representation.

Tokens ...
Context Vector →

3. Calculate Next-Token Probabilities

The model computes a probability distribution over the entire vocabulary.

blue	0.45
clear	0.20
bright	0.10
...	...
other	0.25

4. Apply Decoding Strategy

A decoding strategy (e.g., greedy, beam search, sampling) is applied to the probabilities.

Strategies

- Greedy Decoding
- Beam Search
- Sampling-Based Decoding

5. Select Next Token

A token is selected based on the chosen decoding strategy.

Example:
Selected token → "blue"

6. Repeat Process

The selected token is added to the context, and the process repeats for the next token.

The | sky | is | blue | ?

7. Generated Text

The process continues until a stopping criterion is met (e.g., end token or max length).

Example Output:
"The sky is blue and clear."

Design an AI Text Generator

- **Chatbot**
- What is the input?
- What tokens will the model predict?
- Which decoding strategy would you choose?

Lec#6

Classical Neural Generative Models: Autoencoders, VAEs, and GANs

- **Learning Objectives**
- By the end of this lecture, students will be able to:
- Understand the basic principles of neural generative models.
- Explain the concept of latent space and representation learning.
- Describe the working principles of Autoencoders, VAEs, and GANs.
- Compare classical generative models with modern approaches such as LLMs.
- Identify the strengths and limitations of classical generative models.

- **What Are Neural Generative Models?**
- **Definition**
- Neural generative models are deep learning models that learn patterns from existing data and generate new samples with similar characteristics.
- **Main Idea**
- **Data → Learn hidden patterns → Generate new data**

What Are Neural Generative Models?

DEFINITION

Neural generative models are deep learning models that learn patterns from existing data and generate new samples with similar characteristics.

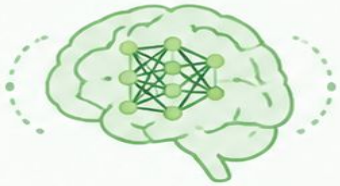
MAIN IDEA

DATA



Existing data

LEARN HIDDEN PATTERNS



Model learns the underlying patterns and structure

GENERATE NEW DATA

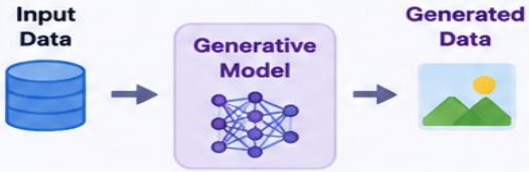


New samples with similar characteristics

EXAMPLES / APPLICATIONS

- Generate images
- Create synthetic data
- Learn meaningful representations

HOW IT WORKS



The model learns the data distribution and generates new, similar data.

EXAMPLES OF MODELS

- Autoencoders (AE)
- Variational Autoencoders (VAE)
- Generative Adversarial Networks (GANs)

KEY TAKEAWAY:
Neural generative models learn from data and can create new, realistic samples.
They are the foundation of many modern AI applications.

Concept of Latent Space

What is Latent Space?

Latent space is a compressed representation where a model stores important features of data.

Example: Image of a face

Original Image



Latent Representation

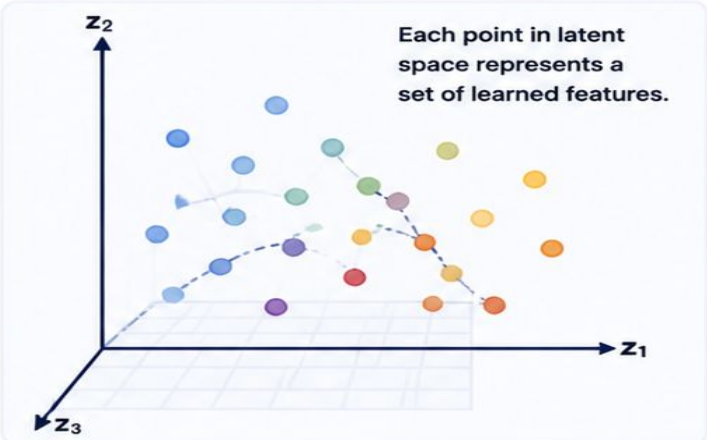


Features



Generated Face





Key Idea

Models learn hidden structures instead of memorizing data.



Activity : Discover Latent Features

- **Think Like a Neural Network**
- Input:
- Images of different animals:
-  Cat
   Dog
   Tiger
- **What hidden features should the model learn?**